

DAGE Manuscript

DAGE

Neural Network Classification of Open-Text Survey Responses

Introduction

The goal of this project was to determine if neural network models could successfully predict a person's survey outcomes using solely their responses from open-text questions. Two separate models were developed, each predicting a unique survey outcome from the respective open-text question used as inputs. The two models are:

1. Predicting 'dominant transportation method' from the open-text responses to question:

"Describe your ideal way to get around your city, and why you prefer it.

To help make your response has richly educational as possible for our students we request you consider your response to offer a thoughtful and meaningful entry."

2. Predicting 'dominant apprehension type' from the open-text responses to question:

"If a friend told you they get nervous talking to new people, what advice would you give them?

To help make your response has richly educational as possible for our students we request you consider your response to offer a thoughtful and meaningful entry."

Defining Outcome Variables

The outcome variables for both of our models were engineered to enable each model to attempt a binary classification. This section will detail the process for creating these categories and assigning classes to all respondents.

Dominant Transportation Method

Dominant Transportation is an engineered outcome variable that identifies the greatest of two interactive scores (if any): Estimated monthly rideshare usage & estimated monthly public transit usage.

```
#| eval: FALSE
#| echo: TRUE
# Set numeric values for transportation variables
usage_monthly <- c(
  "Never" = 0,
  "0-1 days a month" = 1,
  "2-4 days a month" = 3,
  "4-8 days a month" = 6,
  "8 or more days a month" = 8
)
usage_daily <- c(
  "1-2 rides in a typical day" = 1.5,
  "3-4 rides in a typical day" = 3.5,
  "5-6 rides in a typical day" = 5.5,
  "7 or more rides in a typical day" = 7
)

# Calculate highest usage transport method
survey <- survey |>
  mutate(

#   Convert 3-month usage variables to numeric
  across(c(q26_public_transit_3months, q28_rideshare_3months), ~ usage_monthly[.x]),

#   Convert typical daily usage variables to numeric
  across(c(q27_public_transit_dayuse, q29_rideshare_dayuse), ~ usage_daily[.x]),

#   Multiply usage over past 3 months by typical daily usage (usage count estimate)
  public_transit_count = q26_public_transit_3months * q27_public_transit_dayuse,
  rideshare_count = q28_rideshare_3months * q29_rideshare_dayuse,

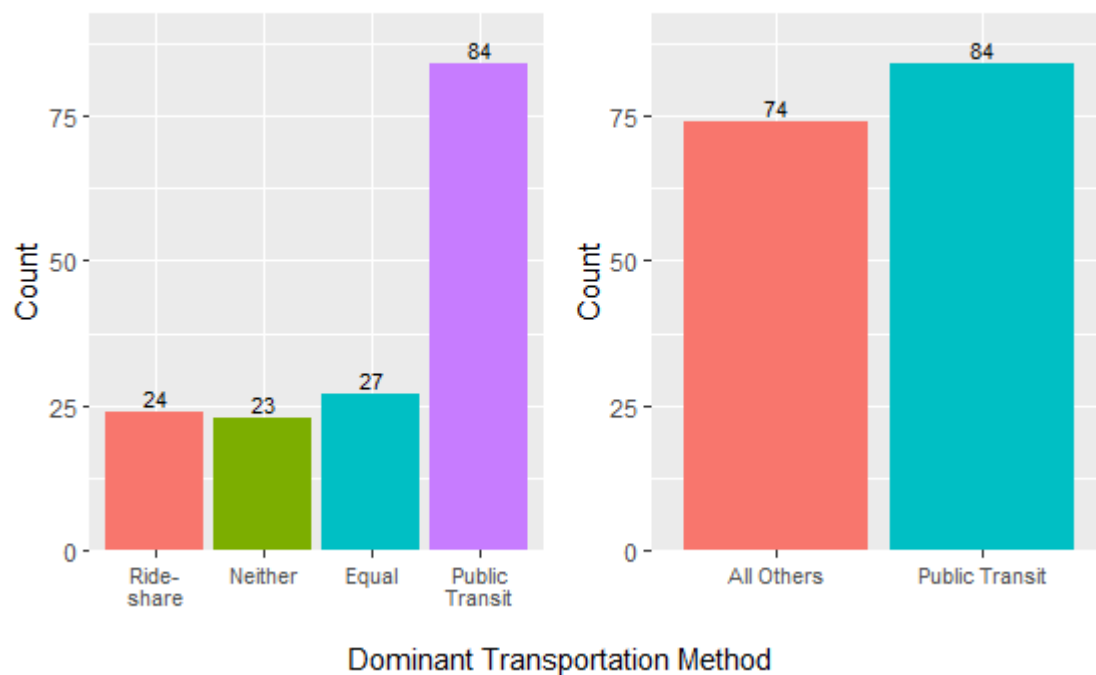
#   Assign highest usage transport method (equal if BA)
  highest_transport = case_when(
    public_transit_count > rideshare_count ~ "public_transit",
    rideshare_count > public_transit_count ~ "rideshare",
    public_transit_count == 0 & rideshare_count == 0 ~ "neither",
```

```
.default = "equal" )
)
```

After numericizing values from a participants' survey responses, relevant questions were sorted into these two groups, depending on which transportation method each question targeted. Each group then had the corresponding values multiplied to estimate the number of monthly trips for both transportation methods. The Dominant Transportation Method was then assigned one of four values.

- If both scores have a value of 0, a participant is assigned to the 'neither' category (indicated no rideshare or public transit use)
- If both scores have a value > 0 , but are still equal, a participant is assigned to the 'equal' category (indicated equal usage of both methods)
- If there was a greater value between estimated rideshare usage and estimated public transit usage, a participant was assigned to the category with higher usage.

Dominant Transportation Method is Imbalanced



The distribution of Dominant Transportation Methods was grossly imbalanced when considering all possible categories (rideshare, public transit, neither, equal), however the distribution became roughly balanced when collapsing categories down to public transit vs. all other possibilities combined. Our group elected to collapse categories as such in order to enable a binary classification approach using a relatively balanced sample.

NOTE: The engineering of 'Dominant Transportation Method' occurs in the `clean_data.qmd` script.

NOTE: Rendering of the above figures occurs in the `eda_dependents.qmd` script.

Dominant Apprehension Type

Dominant Apprehension Type is an engineered outcome variable that identifies the greatest of two composite scores: Group Discussion Apprehension & Solo Conversation Apprehension.

```
#| eval: FALSE
#| echo: TRUE
# Identify forward and reverse-coded Q's (Higher scores = more apprehensive)
forward_code <- c(
  "q1_dislike_groupdiscussion", "q3_nervous_groupdiscussion",
  "q5_tense_newppl_groupdiscussion", "q13_nervous_newppl_conv",
  "q15_nervous_conv", "q18_afraid_conv"
)
reverse_code <- c(
  "q2_comfort_groupdiscussion", "q4_like_groupdiscussion",
  "q6_relaxed_groupdiscussion", "q14_nofear_speaking_conv",
  "q16_relaxed_conv", "q17_relaxed_newppl_conv"
)
# Set numeric values for Likert scales
likert_forward <- c(
  "Strongly disagree"      = 1,
  "Somewhat disagree"     = 2,
  "Neither agree nor disagree" = 3,
  "Somewhat agree"        = 4,
  "Strongly agree"        = 5
)
likert_reverse <- c(
  "Strongly disagree"      = 5,
  "Somewhat disagree"     = 4,
  "Neither agree nor disagree" = 3,
  "Somewhat agree"        = 2,
  "Strongly agree"        = 1
)
# Convert to numeric
survey <- survey |>
  mutate(
    across(all_of(forward_code), ~ likert_forward[.x]),
```

```

    across(all_of(reverse_code), ~ likert_reverse[.x])
  )
# Identify group discussion vs. solo conversation Q's
group_discussion <- c(
  "q1_dislike_groupdiscussion", "q2_comfort_groupdiscussion",
  "q3_nervous_groupdiscussion", "q4_like_groupdiscussion",
  "q6_relaxed_groupdiscussion"
)
solo_conversation <- c(
  "q13_nervous_newppl_convo", "q14_nofear_speaking_convo",
  "q15_nervous_convo", "q16_relaxed_convo",
  "q17_relaxed_newppl_convo", "q18_afraid_convo"
)
# Calculate biggest fear
survey <- survey |>
  mutate(

#   Average of group apprehension Q's
  group_apprehension = rowMeans(
    pick(all_of(group_discussion)), na.rm = TRUE),

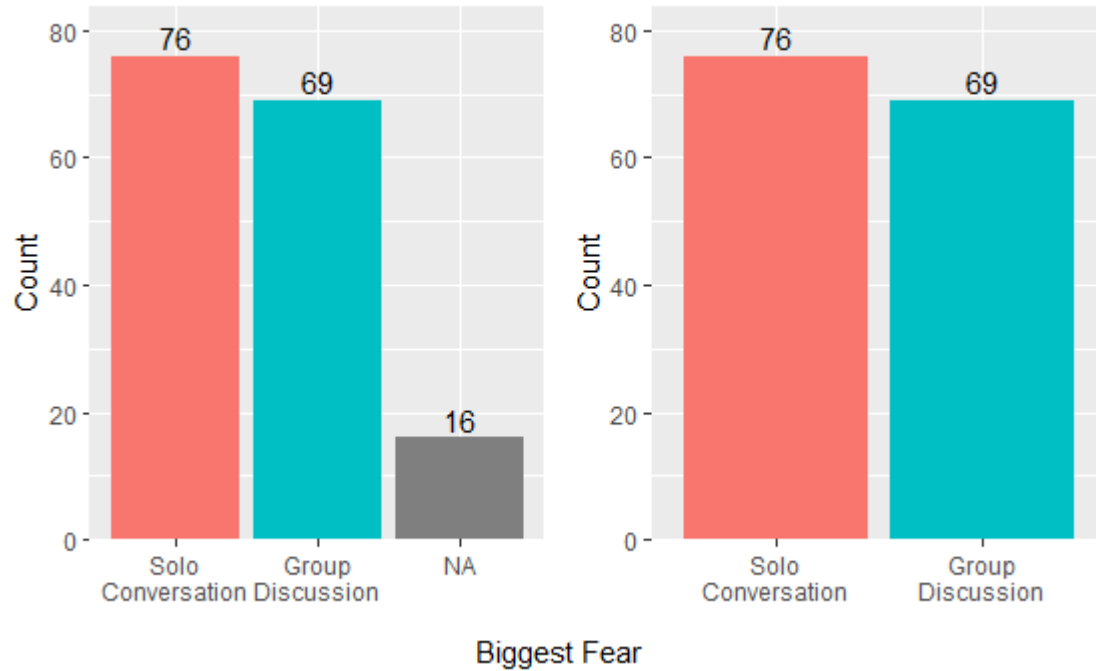
#   Average of solo apprehension Q's
  solo_apprehension = rowMeans(
    pick(all_of(solo_conversation)), na.rm = TRUE),

#   Assign greatest of the two values (equal becomes NA)
  biggest_fear = case_when(
    group_apprehension > solo_apprehension ~ "group_apprehension",
    solo_apprehension > group_apprehension ~ "solo_apprehension",
    .default = "equal")
  )

```

After numericizing Likert scale values from a participants PRCA responses, relevant questions were sorted into these two groups, depending on which type of apprehension each question targeted. Each group then had the corresponding numeric Likert scale values averaged to calculate the composite scores for ‘average group discussion apprehension’ and ‘average solo conversation apprehension’. Finally, the Dominant Apprehension Type takes the largest of these two values and codes for the corresponding group, or indicates if both scores are equal.

Dominant Apprehension Type is Roughly Equal



The distribution of dominant apprehension types was grossly imbalanced across all possible categories (group discussions, solo conversations, and equal apprehension) but becomes roughly balanced when excluding the equal apprehension group. Our group elected to exclude equal apprehension responses to enable a binary classification approach with a relatively balanced sample.

NOTE: The engineering of 'Dominant Transportation Method' occurs in the `clean_data.qmd` script.

NOTE: Rendering of the above figures occurs in the `eda_dependents.qmd` script.

Tools

The following are a list of tools utilized throughout the project:

- R was used primarily for cleaning and EDA scripts prior to building the neural networks
- Python was utilized for building the neural networks and visualizing results, primarily relying on the TensorFlow, Keras, and Seaborn packages. Agentic IDEs including Devin and Google Colab supported this process.
- Quarto was utilized for creating research memos and the final manuscript.

- A GitHub repository was utilized by the team to manage collaboration and development history.
- A markdown file (contributions.md) was kept to record individual team member contributions to the project.

Methods

Both models utilized similar neural network models to accomplish their tasks, with slight differences in model architectures and hyperparameter values that were individualized during model tuning.

Model Architectures

With both models taking natural language data as inputs, the first step was to tokenize each vocabulary set, then vectorize participant responses accordingly. Both models tokenized on whole words, employed a maximum vocabulary size of 5000 tokens, and a maximum sequence length of 100 tokens. The vocabulary size limit means that only the 5000 most common words across the training set were given a unique token identifier, with any other rare words being assigned to a placeholder token value indicating that this word was not included in the vocabulary. The sequence length limit means that any responses longer than 100 words were truncated.

Vectorized responses for each model are then passed into an embedding layer, which through training will learn dense numeric vector representations for each unique token. These embeddings allow the network to capture statistical relationships between words, resulting in words that contribute similarly to classification developing similar embedding vectors. These associations may emerge among words that are more unique to either solo conversation apprehension or group discussion apprehension, for example.

Each full sequence of word embeddings proceeds to a global average pooling layer that compresses the expanded embedding sequence back down into a single vector representation of the response. Global average pooling means that each embedding dimension is averaged across all 100 token positions in the sequence, discarding information regarding word order to produce a compact vector representation that summarizes what ‘kinds’ of words appear in each response overall. Where the original text vector only contained discrete token indices, the pooled representation has now encoded information that was captured within the learned embeddings.

Pooled representations then enter a fully connected dense layer using ReLU as the activation function. Each unit here learns a weighted combination of the pooled embedding features, indicating which combinations of lexical or stylistic features are most meaningful for discriminating between classes. ReLU is important here as a non-linear activation function, allowing the network to learn more complex decision boundaries. In short, the dense layer transforms

the pooled representations within feature space towards representations that will be more easily separated by the final output layer.

Following the dense ReLU layer, a dropout layer was applied as a form of regularization during training. The dropout rate (e.g., 0.2), assigns a proportion of units from the dense layer to be randomly deactivated for each training batch. This step protects against the network relying too heavily on any single unit or narrow combination of features. In the context of this natural language data, dropout here effectively protects from behaviors such as the network relying too much on the presence or absence of a small set of keywords or stylistic markers to make accurate classifications, and/or the network strongly memorizing and overfitting to training data with an adverse effect on performance with held-out data. What is encouraged instead is for the network to learn broader patterns that will lead to a greater ability to generalize. Note here that the dropout layer is only active during training, with test set predictions being made with all units remaining active after having ideally benefited from the dropout procedure.

Last is the output layer consisting of one unit using a Sigmoid activation function and sparse categorical cross-entropy loss function to classify responses for each research question. Sigmoid converts the hidden layer representations into a probability that the response belongs in the positive class. Positive and negative classes are arbitrary for our research questions and were randomly assigned, for instance with solo conversation apprehension being assigned as the positive class versus group discussion apprehension being the negative class. The only implication of this decision is that the network is technically just calculating the probability that a response belongs to an individual whose dominant apprehension type is solo conversations, with a conventional threshold of 0.5. If the output layer produces a value above 0.5, a response is classified as solo-conversation apprehension dominant, and group-discussion apprehension dominance is assigned for probabilities below 0.5.

Information Flow Through the Network

Under the guidance of this explanation, consider now how a test pattern may flow through the fully trained network, shifting to an example from the transportation methods model:

A text response is passed into the vectorization layer and numericized. “I love riding the bus” becomes a vector such as [47, 1, 33, 124, 1, 0, ... 0] with a unique value representing each retained word. The Keras TextVectorization layer assigns values of 1 to any word excluded from the restricted vocabulary size of 10,000. Values of 0 are used as padding between the last word and remaining positions before reaching the fixed sequence length of 100 tokens.

When the vectorized response passes through the embedding layer, each token integer is converted into an associated embedding vector to create a dense representation of the response. For example, the word assigned an index value of 42 will now become its own multi-dimensional vector, ex: [0.42, -0.37, 0.06, 0.94, ..., 0.04]. These embeddings are the learned representations that encode relationships between words with similar usage patterns and related stylistic/lexical features. Tokens for related words such as “bus”, “train”, and “subway”, for instance, may

now occupy nearby regions in embedding space. The verse itself has now expanded from a one-dimensional vector of sequence length 100 into a two-dimensional tensor, similar in structure to a feature matrix.

The global average pooling layer now performs a relatively simple operation. For all embedding dimensions, this layer computes the mean value across all token positions. The two-dimensional tensor is thereby compressed back down to a one-dimensional vector containing the average activation of each embedding dimension throughout the response.

The purpose of the dense layer is to reorganize this representation in feature space to make it more easily separable into the binary classes. The pooled embedding features are transformed through weighted combinations the network learns are most informative for classification. The resulting vector is then fed into the output layer.

Lastly, the output layer uses this transformed representation to generate the probabilities for the positive class, as discussed earlier. During training, dropout would also have been applied with a proportion of the dense layer units deactivated, and prediction error would backpropagate through the network to update weights throughout the embedding, dense and output layers.

Behavior and Performance

The behavior of interest for this project is the categorical prediction of unseen verses by the fully trained network, with performance being measured in terms of accuracy. Because our classes are slightly imbalanced for each research question, accuracy is a somewhat inflated metric of model performance, however its native support in our Tensorflow/Keras workflow led our group to elect to continue using this metric while acknowledging its limitations.

Results

Dominant Transportation Methods

The binary neural network predicted whether participants were public-transit dominant or belonged to the combined “other” category. The other category included rideshare-dominant participants, participants with equal estimated rideshare and public-transit use, and participants with no estimated use of either option. The model used a classification threshold selected from the validation data by maximizing accuracy.

```
import numpy

import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
```

```
# Run the transportation model and retain its results.
transport_results = runpy.run_path(
    "scripts/nn_trans/nn_publicvsotter.py"
)

history = transport_results["history"]
test_accuracy = transport_results["test_accuracy"]
test_precision = transport_results["test_precision"]
test_recall = transport_results["test_recall"]
test_f1 = transport_results["test_f1"]
best_threshold = transport_results["best_threshold"]
test_confusion_df = transport_results["test_confusion_df"]
```

```
accuracy_history = pd.DataFrame(
    {
        "Epoch": range(
            1,
            len(history.history["accuracy"]) + 1,
        ),
        "Training accuracy": history.history["accuracy"],
        "Validation accuracy": history.history["val_accuracy"],
    }
)

accuracy_history = accuracy_history.melt(
    id_vars="Epoch",
    var_name="Dataset",
    value_name="Accuracy",
)

sns.lineplot(
    data=accuracy_history,
    x="Epoch",
    y="Accuracy",
    hue="Dataset",
)

plt.ylim(0, 1)
plt.xlabel("Training epoch")
plt.ylabel("Accuracy")
plt.tight_layout()
```

```
plt.show()
```

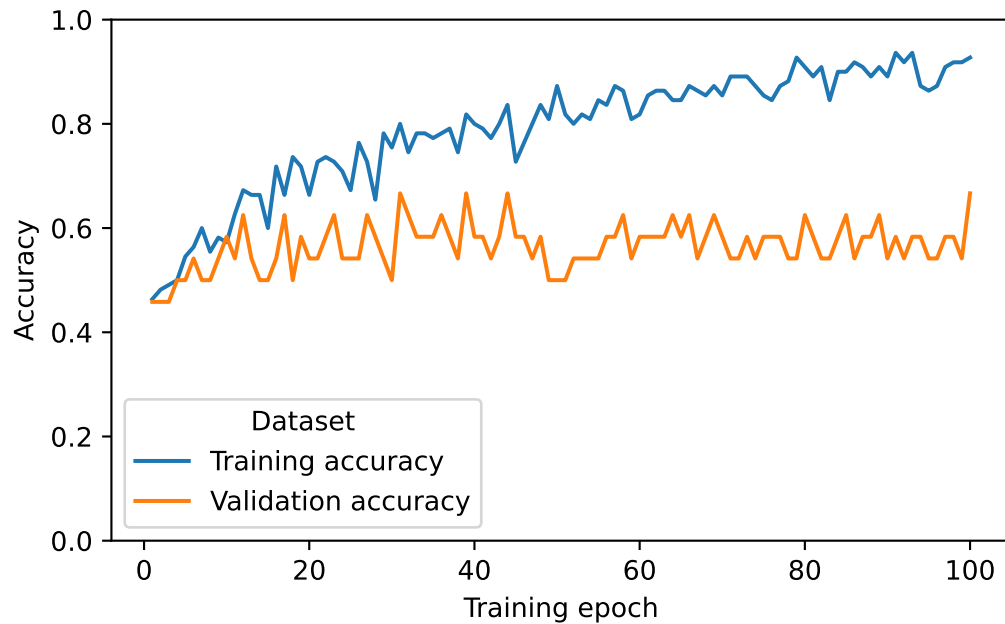


Figure 1: Training and validation accuracy across neural-network training epochs.

As shown in Figure 1, training and validation accuracy were tracked across epochs to evaluate the model's learning behavior.

```
performance_results = pd.DataFrame(  
    {  
        "Metric": [  
            "Classification threshold",  
            "Accuracy",  
            "Public-transit precision",  
            "Public-transit recall",  
            "Public-transit F1",  
        ],  
        "Value": [  
            best_threshold,  
            test_accuracy,  
            test_precision,  
            test_recall,  
            test_f1,  
        ],  
    })
```

```

    ],
  }
)

performance_results["Value"] = (
    performance_results["Value"]
    .round(3)
)

performance_results

```

Table 1: Final test-set performance for the public-transit versus other neural network.

	Metric	Value
0	Classification threshold	0.450
1	Accuracy	0.792
2	Public-transit precision	0.750
3	Public-transit recall	0.923
4	Public-transit F1	0.828

As reported in Table 1, the model achieved an accuracy of 0.792 and a public-transit F1 score of 0.828 on the held-out test set. Public-transit precision was 0.75, while public-transit recall was 0.923. Predictions were classified using a validation-selected threshold of 0.45.

```

sns.heatmap(
    test_confusion_df,
    annot=True,
    fmt="d",
    cbar=False,
)

plt.xlabel("Predicted transportation category")
plt.ylabel("Actual transportation category")
plt.tight_layout()
plt.show()

```

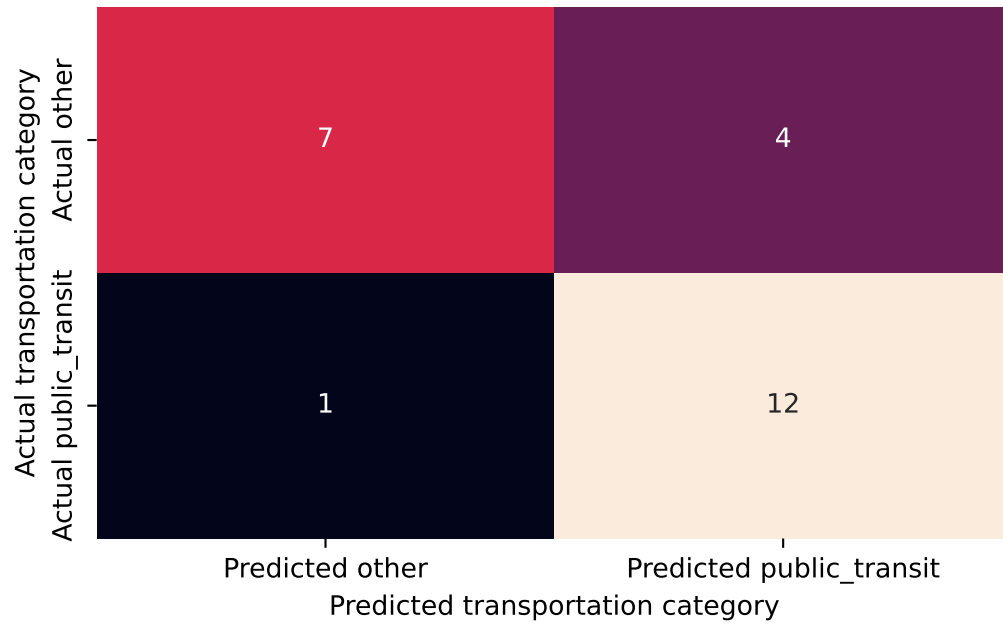


Figure 2: Confusion matrix for public-transit versus other predictions on the held-out test set.

Figure 2 shows the number of correct and incorrect classifications for each outcome. Cells along the diagonal represent correct predictions, while the off-diagonal cells represent cases that the model misclassified. This provides additional context beyond overall accuracy by showing whether errors were concentrated among public-transit-dominant participants or participants in the combined other category.

Dominant Apprehension Types

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
history = pd.read_csv("results/training_history_apprehension.csv")
```

```
# Create epoch numbers starting at 1
epochs = range(1, len(history) + 1)

# Plot training and validation loss
plt.plot(
    epochs,
```

```

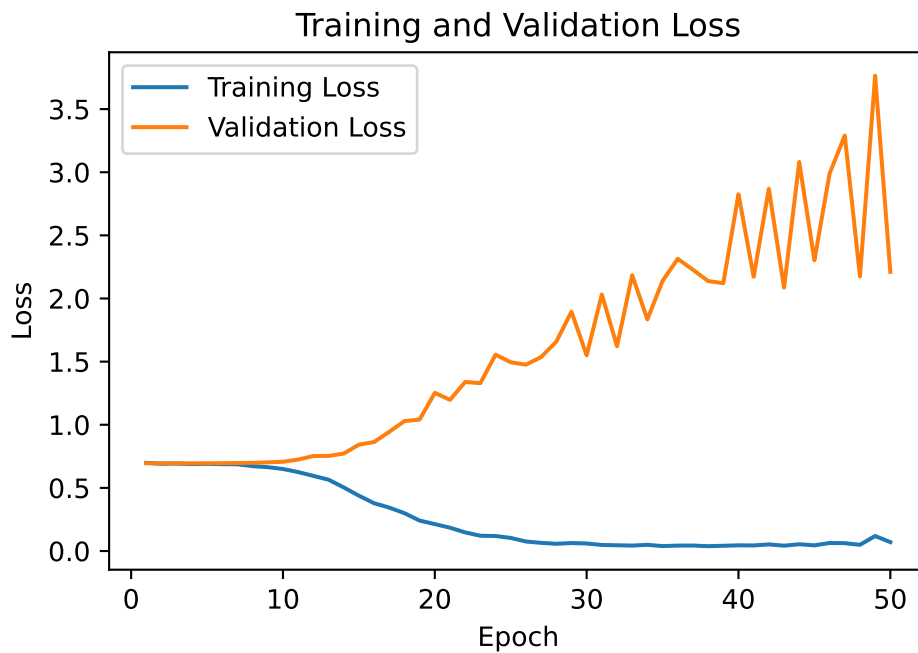
    history["training_loss"],
    label="Training Loss"
)

plt.plot(
    epochs,
    history["validation_loss"],
    label="Validation Loss"
)

# Add labels and title
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training and Validation Loss")

# Display legend and plot
plt.legend()
plt.show()

```



```

# Create epoch numbers starting at 1
epochs = range(1, len(history) + 1)

```

```

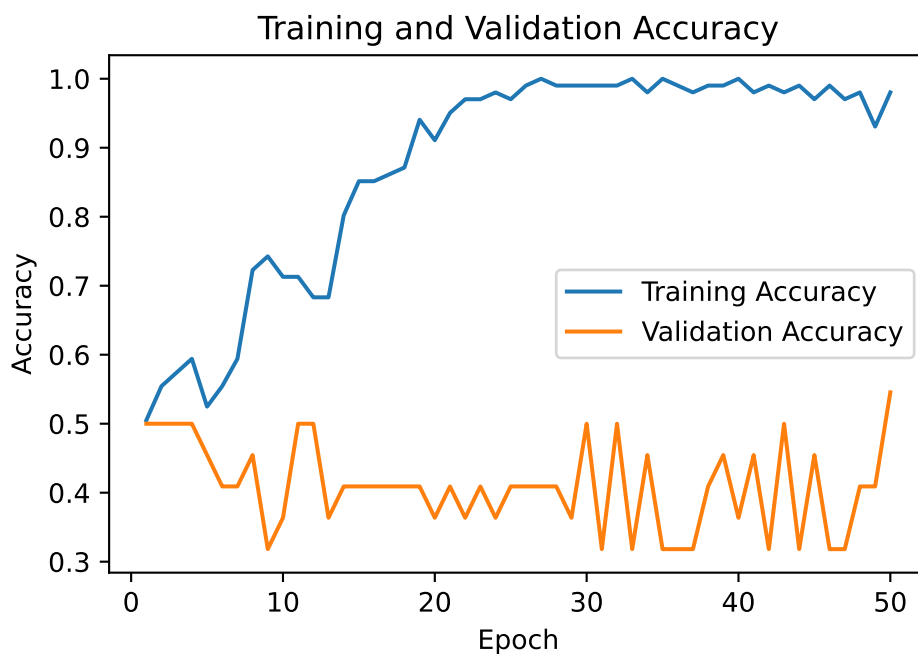
# Plot training and validation loss
plt.plot(
    epochs,
    history["training_accuracy"],
    label="Training Accuracy"
)

plt.plot(
    epochs,
    history["validation_accuracy"],
    label="Validation Accuracy"
)

# Add labels and title
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training and Validation Accuracy")

# Display legend and plot
plt.legend()
plt.show()

```



While the transportation methods model was able to learn a meaninnful decision boundary

with satisfying accuracy, the apprehension types model struggled. Although both models shared similar architecture, inputs, and sample sizes, the apprehension model demonstrated a propensity to immediately overfit to training data in all training runs. This model is essentially reverting to a strategy of memorizing training patterns after not being able to find strong relationships between inputs and outputs in the validation set.

```
import random
import numpy as np
import tensorflow as tf
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, classification_report,
import seaborn as sns
import torch
from pathlib import Path
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from tensorflow import keras
from tensorflow.keras import layers

SEED = 42
path_data = Path("results")
le = LabelEncoder()

survey = pd.read_csv("data_clean/survey_apprehension.csv")

# Extract out "Equal"/NA observations
survey_na = survey[survey['biggest_fear'].isna()].copy()

# Keep only rows where biggest_fear is not NA
survey = survey[survey['biggest_fear'].notna()].copy()

# Create splits on non-equal observations
survey_train, survey_temp = train_test_split(
    survey,
    test_size=0.30,
    stratify=survey["biggest_fear"],
    random_state=SEED
)
survey_val, survey_test = train_test_split(
    survey_temp,
    test_size=0.50,
    stratify=survey_temp["biggest_fear"],
    random_state=SEED
)
```



```

x_train = survey_train["q_advice_text"].astype(str).to_numpy()
x_val = survey_val["q_advice_text"].astype(str).to_numpy()
x_test = survey_test["q_advice_text"].astype(str).to_numpy()

y_train = le.fit_transform(survey_train["biggest_fear"])
y_val = le.transform(survey_val["biggest_fear"])
y_test = le.transform(survey_test["biggest_fear"])

num_classes = len(le.classes_)
print(f'Classes: {le.classes_}')

# Parameters for vectorization
max_tokens = 10000
sequence_length = 100

vectorizer = layers.TextVectorization(
    max_tokens=max_tokens,
    output_mode='int',
    output_sequence_length=sequence_length,
)

# Adapt vectorizer to training text
vectorizer.adapt(x_train)
# Transform inputs to vectors
x_train = vectorizer(x_train)
x_val = vectorizer(x_val)
x_test = vectorizer(x_test)

tf.random.set_seed(SEED)

model = keras.Sequential([
    # Specify that each input is a sequence of max_length token IDs
    layers.Input(shape=(max_tokens,)),

    # Embedding layer maps vocabulary tokens to 64-dimensional vectors
    layers.Embedding(input_dim=max_tokens, output_dim=64),

    # Average embeddings across the sequence length
    layers.GlobalAveragePooling1D(),

    # Fully connected layers

```

```

        layers.Dense(128, activation='relu'),
        layers.Dropout(0.2),

        # Output layer
        layers.Dense(num_classes, activation='sigmoid')
    ])

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'

model.load_weights(path_data/"apprehension_model.weights.h5")

# Generate predicted probabilities
y_pred_probs = model.predict(x_val, verbose=0)

# Convert predicted probabilities to class indices
y_pred = np.argmax(
    np.asarray(y_pred_probs, dtype=np.float32),
    axis=1
)

# Ensure true labels are integer class indices
y_true = np.asarray(y_val, dtype=np.int64)

```

Classes: ['group_apprehension' 'solo_apprehension']

WARNING:tensorflow:5 out of the last 9 calls to <function TensorFlowTrainer.make_predict_fun

C:\Users\Andrew\Documents\GitHub\DAGE_andrew\.venv\Lib\site-packages\keras\src\saving\saving
 saveable.load_own_variables(store)

```

# Create confusion matrix
cm = confusion_matrix(y_true, y_pred)

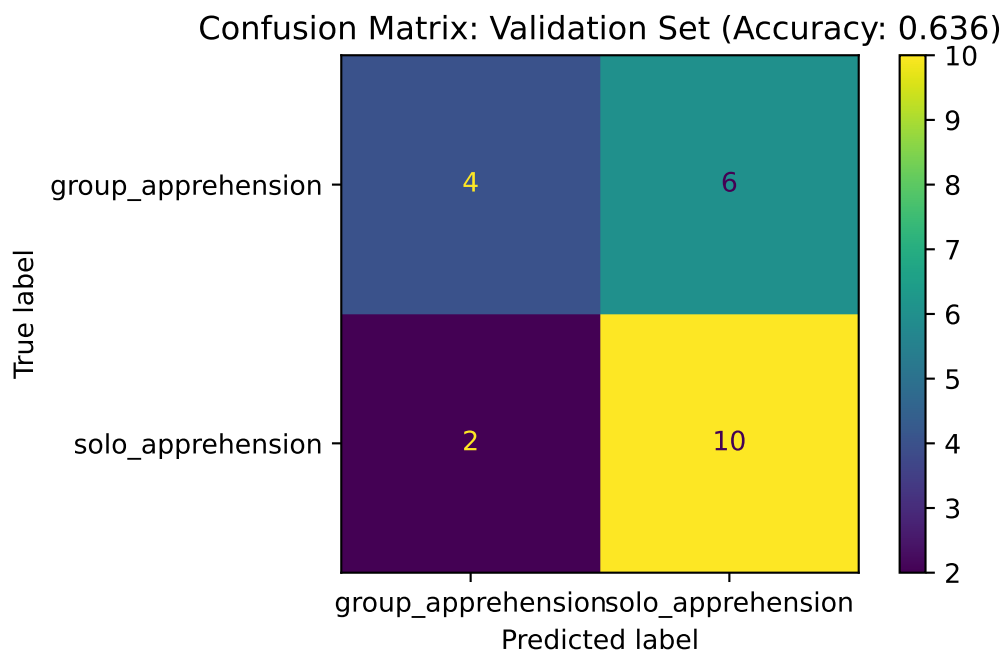
# Calculate accuracy from confusion matrix
val_accuracy = cm.diagonal().sum() / cm.sum()

```

```
# Plot
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=le.classes_
)

disp.plot()

plt.title(f"Confusion Matrix: Validation Set (Accuracy: {val_accuracy:.3f})")
plt.show()
```



```
# Generate predicted probabilities
y_pred_probs = model.predict(x_test, verbose=0)

# Convert predicted probabilities to class indices
y_pred = np.argmax(
    np.asarray(y_pred_probs, dtype=np.float32),
    axis=1
)

# Ensure true labels are integer class indices
y_true = np.asarray(y_test, dtype=np.int64)
```

```

# Create confusion matrix
cm = confusion_matrix(y_true, y_pred)

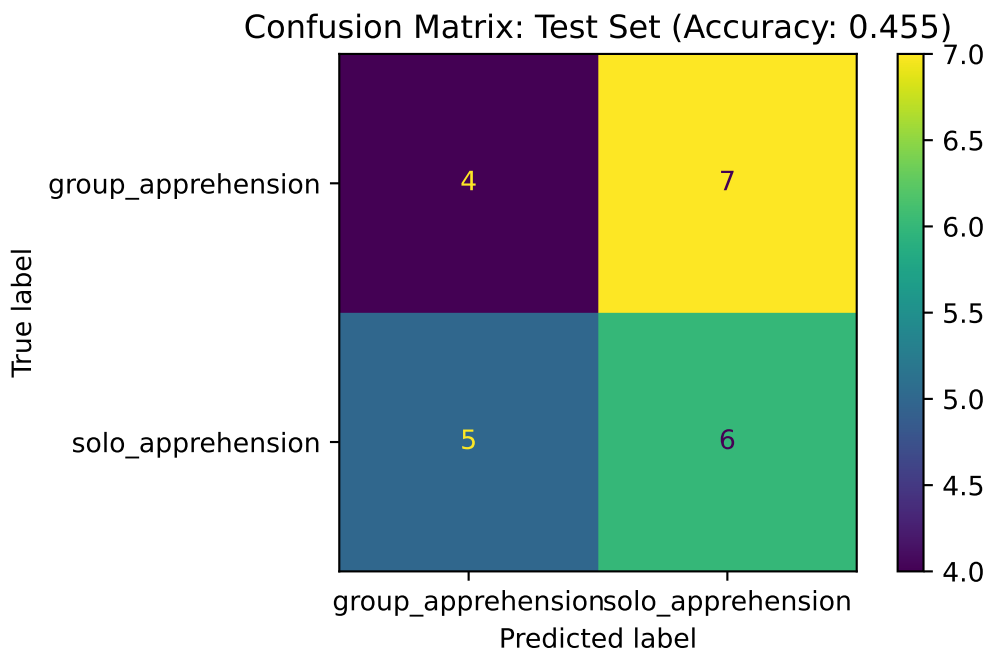
# Calculate accuracy from confusion matrix
test_accuracy = cm.diagonal().sum() / cm.sum()

# Plot
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=le.classes_
)

disp.plot()

plt.title(f"Confusion Matrix: Test Set (Accuracy: {test_accuracy:.3f})")
plt.show()

```



Although the model achieves performance in the validation set of 0.636 that is a slight improvement from a mean-only model, this performance increase vanishes when the test set is evaluated (accuracy: 0.455). The apprehension types model ultimately failed to learn a meaningful decision boundary, where the transportation methods model succeeded. The

analysis section will investigate the performance of the transportation methods model in an attempt to explain the disparate outcomes across our two research questions.

Analysis

Ablation test

We did an analysis of the most common words within all of the classes described by Andrew's memo. The results are shown in the following table.

```
import re
from collections import Counter

import numpy as np
import pandas as pd
from IPython.display import Markdown, display

DATA_PATH = "data_clean/survey_transportation.csv"

STOPWORDS = {
    'a', 'about', 'all', 'also', 'am', 'an', 'and', 'are', 'as', 'at', 'be', 'been', 'being', 'but', 'by',
}

USAGE_MONTHLY = {
    'Never': 0,
    '0-1 days a month': 1,
    '2-4 days a month': 3,
    '4-8 days a month': 6,
    '8 or more days a month': 8,
}

USAGE_DAILY = {
    '1-2 rides in a typical day': 1.5,
    '3-4 rides in a typical day': 3.5,
    '5-6 rides in a typical day': 5.5,
    '7 or more rides in a typical day': 7,
}

TRANSPORT_COLS = ['Q26', 'Q27', 'Q28', 'Q29']

def tokenize(text):
    if pd.isna(text):
        return []
```

```

text = str(text).lower()
text = re.sub(r'[\w\s]', ' ', text)
return [t.strip(" ") for t in text.split() if t.strip(" ")]

def extract_classes(df):
    df = df.copy()
    df['q26_num'] = df['Q26'].map(USAGE_MONTHLY)
    df['q27_num'] = df['Q27'].map(USAGE_DAILY)
    df['q28_num'] = df['Q28'].map(USAGE_MONTHLY)
    df['q29_num'] = df['Q29'].map(USAGE_DAILY)
    df['public_transit_count'] = df['q26_num'] * df['q27_num']
    df['rideshare_count'] = df['q28_num'] * df['q29_num']
    trans_cond = [
        df['public_transit_count'] > df['rideshare_count'],
        df['rideshare_count'] > df['public_transit_count'],
    ]
    trans_choices = ['Public Transit', 'Rideshare']
    df['highest_transport'] = np.select(trans_cond, trans_choices, default='Equal')
    return df

df = pd.read_csv(DATA_PATH)
df = extract_classes(df)
tr_df = df[df["Q19"].notna() & df[TRANSPORT_COLS].notna().all(axis=1)].copy()

for cls in sorted(tr_df["highest_transport"].dropna().unique()):
    tokens = tr_df.loc[tr_df["highest_transport"] == cls, "Q19"].astype(str).apply(tokenize)
    all_words = [t for toks in tokens for t in toks]
    content_words = [w for w in all_words if w not in STOPWORDS]
    counter = Counter(content_words)
    top = counter.most_common(10)
    n_cases = len(tokens)
    table_df = pd.DataFrame(top, columns=["Word", "Frequency"])
    table_df.insert(0, "Rank", range(1, len(table_df) + 1))
    header = "| " + " | ".join(table_df.columns) + " |"
    sep = "|" + "|".join(["---" for _ in table_df.columns]) + "|"
    rows = []
    for _, row in table_df.iterrows():
        rows.append("| " + " | ".join(str(v) for v in row) + " |")
    table_md = "\n".join([header, sep] + rows)

```

```
display(
    Markdown(
        f"**{cls} (n = {n_cases})*\n\n" + table_md
    )
)
```

As described in this table the words most indicative of the transportation class are the words “bus” and “walking” for the public transportation class.

Model comparison: with vs. without “bus” and “walking”

The two scripts below train the same binary public-transit-vs-other neural network. The first uses the raw responses; the second removes the words “bus” and “walking” before training to measure their contribution.

- scripts/nn_trans/nn_publicvsotter.py
- scripts/nn_publicvsotter_no_bus_walking.py

```
ablation_results = runpy.run_path(
    "scripts/nn_publicvsotter_no_bus_walking.py"
)
```

```
comparison_results = pd.DataFrame(
    {
        "Model": ["Original", "No bus/walking"],
        "Test accuracy": [
            transport_results["test_accuracy"],
            ablation_results["test_accuracy"],
        ],
        "Test precision": [
            transport_results["test_precision"],
            ablation_results["test_precision"],
        ],
        "Test recall": [
            transport_results["test_recall"],
            ablation_results["test_recall"],
        ],
        "Test F1": [
            transport_results["test_f1"],
            ablation_results["test_f1"],
        ],
    }
)
```

```

    }
)

comparison_results.round(3)

```

Table 2: Test-set performance comparison for the original model and the no-bus/walking ablation.

	Model	Test accuracy	Test precision	Test recall	Test F1
0	Original	0.792	0.750	0.923	0.828
1	No bus/walking	0.542	0.542	1.000	0.703

As reported in Table 2, removing the words “bus” and “walking” changes the model’s test-set performance. The drop or change in accuracy and F1 indicates how much the model relied on those two words.

```

fig, ax = plt.subplots(figsize=(6, 4))
ax.bar(
    comparison_results["Model"],
    comparison_results["Test accuracy"],
    color=["#2c5aa0", "#c44e52"],
)
ax.set_ylabel("Test accuracy")
ax.set_title("Public transit vs. other: original vs. ablation")
ax.set_ylim(0, 1)
for i, v in enumerate(comparison_results["Test accuracy"]):
    ax.text(i, v + 0.02, f"{v:.3f}", ha="center")
plt.tight_layout()
plt.show()

```

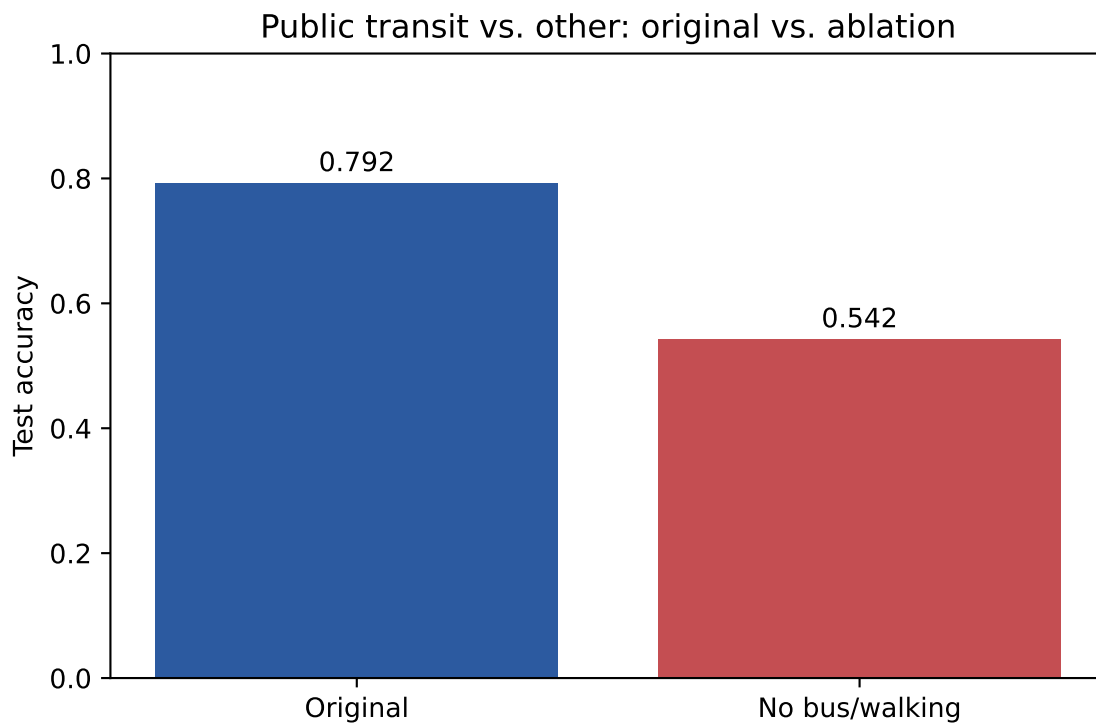



Figure 3: Test-set accuracy comparison for the original model and the no-bus/walking ablation.

Discussion + Next Steps

Our results and analysis have determined that of approach of using open-text data to predict survey outcomes is plausible under the right circumstances. When open-text data is derived from a question that is specifically related to the predicted outcome, with responses that can identify uniquely informative keywords and phrases, a neural network is quickly able to learn these relationships. This phenomenon is clearly demonstrated with the transportation methods model, which effectively learned the relevant keywords for dominant transportation methods. When the open-text data does not provide uniquely informative keywords or phrases, such as with the apprehension types model, the model will likely fail to learn a meaningful decision boundary. Not all questions or survey outcomes are appropriate for this method, although build a neural network NLP workflow is an important consideration for extracting additional insight from survey data.

Next steps would primarily include additional strategies to improve performance of the apprehension model. While tokenizing on words did not prove successful despite extensive tuning and architecture adjustments, tokenizing on bigrams, trigrams, or other approaches may allow the model to learn more complicated semantic information contained within the open-text responses.